

Machine Learning in Practice I

Lecture 2

Antal Jakovac, 2025

Logics of Intelligence



- Consider a world W consisting of 2 pixels
 $\{\square\square, \square\blacksquare, \blacksquare\square, \dots \blacksquare\blacksquare\}$
- W also has two visible states A and B . We observe that
 - $A \rightarrow \{\square\square, \blacksquare\square, \dots \blacksquare\square\}$
 - $B \rightarrow \{\square\blacksquare, \blacksquare\blacksquare, \dots \blacksquare\blacksquare\}$
- **Step1:** problem definition: tell the state from the pixels
→ classification task



- **Step2-3:** Data collection, cleaning, preprocessing
- in our case: observation → numerics
- the pixels are black-and-white → brightness is a number in $[0,1]$
 - black: 0
 - white: 1
 - grey: somewhere in between
- 2 pixels mean 2 numbers (x_1, x_2)
- *usually Step2-3 is rather tedious...*

- **Step4-5-6:** feature engineering, model selection, training (in practice these are separate steps)
- Let us plot the point pairs corresponding to state A and B

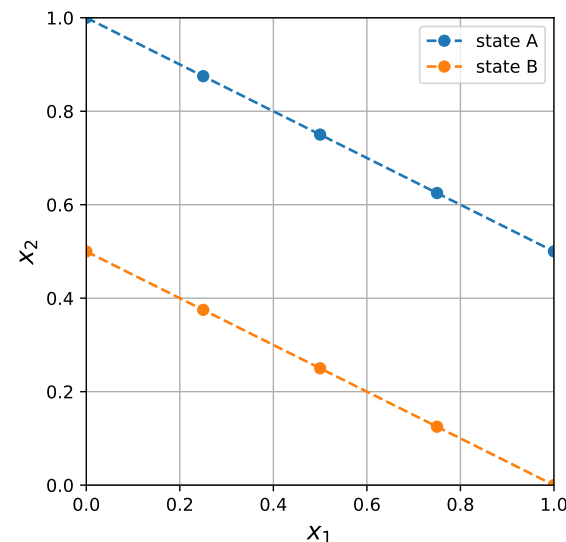
→ the points lie in subspaces (lines)

→ within state A and B

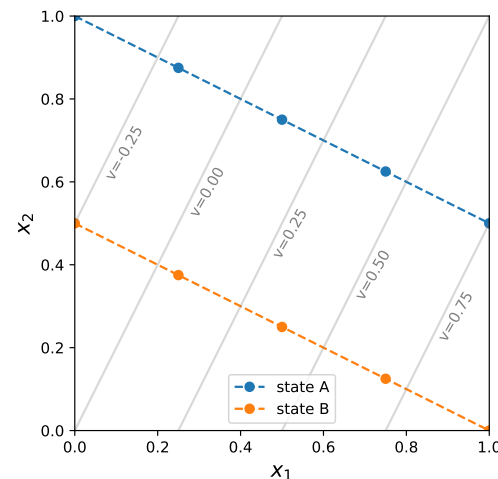
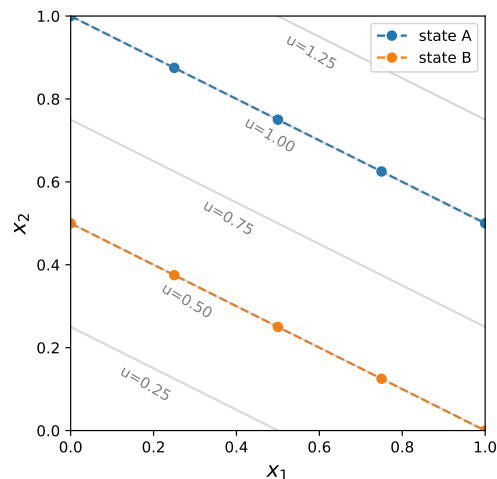
$$\frac{x_1}{2} + x_2 = \begin{cases} 1 & \text{for state A} \\ 0 & \text{for state B} \end{cases}$$

→ worth to introduce new coordinates (features)

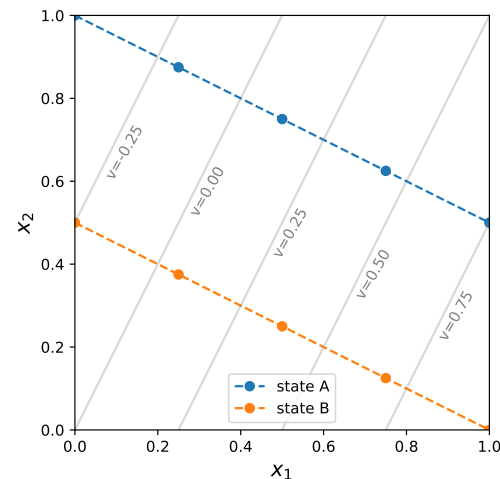
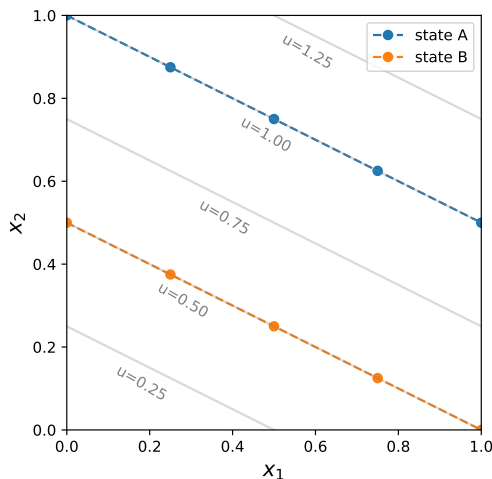
$$u = \frac{x_1}{2} + x_2, \quad v = x_1 - \frac{x_2}{2}$$



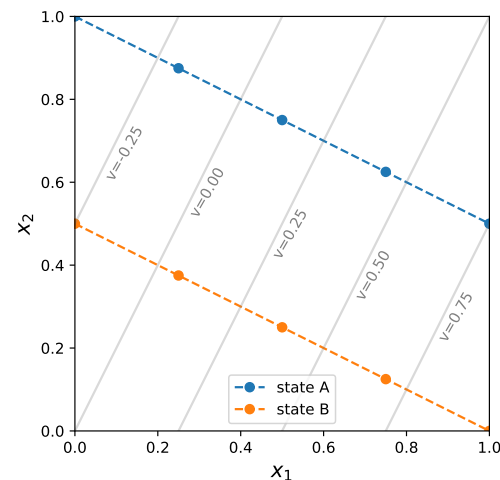
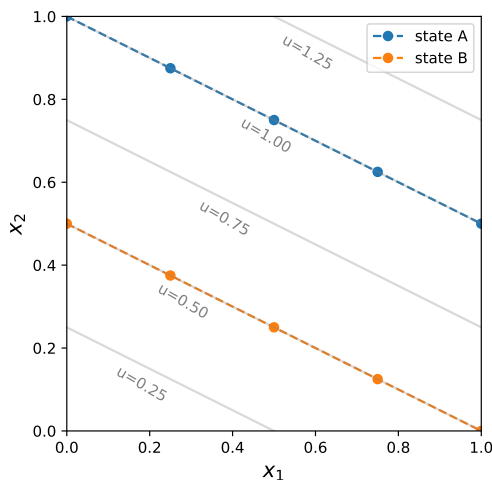
- **Step4-5-6:** feature engineering, model selection, training
- with the new coordinates $x_1 = \frac{2u}{5} + \frac{4v}{5}$, $x_2 = \frac{4u}{5} - \frac{2v}{5}$
- constant u – aligned with states; constant v – changes within a given state



- **Step4-5-6:** feature engineering, model selection, training
- To tell apart state A and B we need only the value of u !
 - ➔ $u = 1.0$ state A
 - ➔ $u = 0.5$ state B
 - ➔ otherwise neither
- solves classification task



- **Step4-5-6:** feature engineering, model selection, training
- To describe a instance belonging to a state we need just v !
 - ➔ state A: v in $[-0.5, 0.75]$
 - ➔ state B: v in $[-0.25, 1]$
- solves compression,
dimensional reduction





- **Step4-5-6:** feature engineering, model selection, training
- a perfect **feature selection** provides a **coordinate system** best aligned with the equivalence classes of the data
 - there are coordinates that are constant on each data class, and have different values on different classes
(*relevant/selective coordinates* – good for classification)
 - there are coordinates that change within a given data class
(*descriptive/irrelevant coordinates* – good for compression)
- **task of all model building is to find (approximately) these coordinates/features**



- **Step4-5-6:** feature engineering, model selection, training
- in practice we separate the task of coordination
 - ➔ **feature selection:** original features, simple combinations
 - ➔ **model selection:** single out a parametrizable functional space to combine the features (distance, linear- or nonlinear combinations)
 - ➔ **training:** determine the free parameters of the model that fits the data the best



- **Step4-5-6:** feature engineering, model selection, training
- **mind vs data?**
 - data/models importance ratio → data are central importance in real world applications
 - are data enough? do we possess the necessary knowledge?
 - present approach: data engineering is the most important, few fundamental research (cf. LeCun vs Wang in Meta)



- **Step7:** evaluation
- try to estimate how well the original task was solved
 - class reconstruction accuracy
 - robustness, proclivity for failures
 - treatment of outliers
 - business benefits



- **Step8:** deployment and monitoring
- in a real-life application the ML code is used as a product
- requires continuous monitoring
 - ➔ data/environment may change
 - ➔ new classes appear
 - ➔ errors may occur



takehome message: Task of the intelligence is to

find the **relevant features** and **adapt** their values to the task

Organizing ML projects

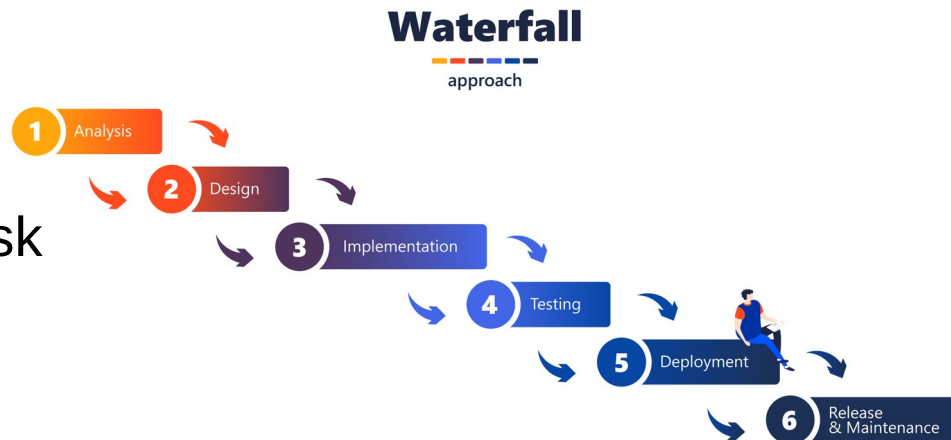
Organizing ML projects

- two popular workflow schemes today: **waterfall** vs. **agile**

- waterfall** workflow:

- clear documentation of the task
- predictable timeline
- inflexible
- late discovery of errors

- good for well-defined projects** (like bridge building)



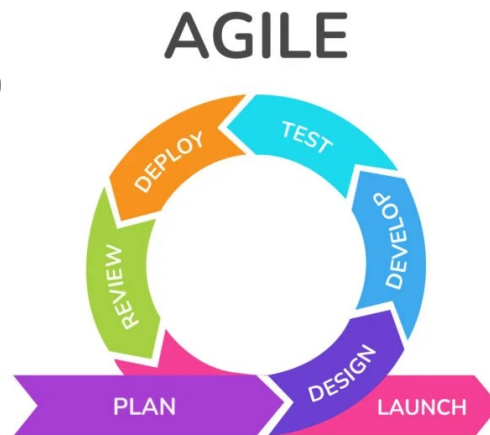
Organizing ML projects

- **agile** workflow: (agile manifesto, description)

- ➔ individuals > processes
 - collaborative vs rigid teams
- ➔ working model > detailed documentation
 - Minimal Viable Product MVP
 - improvement

"If you're not embarrassed by the first version of your product, you've launched too late."

— Reid Hoffman, founder of LinkedIn

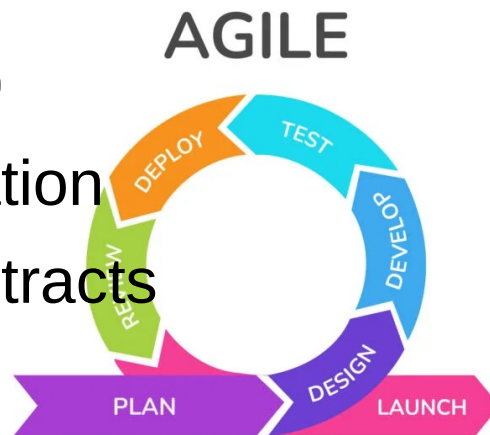


Goal: Solve transportation

- ➔ MVP: Skateboard
- ➔ Next: Bicycle
- ➔ Then: Motorcycle
- ➔ Finally: Car

Organizing ML projects

- **agile** workflow: (**agile manifesto**, **description**)
 - ➔ customer collaboration > contract negotiation
 - customer needs even over signed contracts
 - ➔ responding to change > following a plan
 - original ideas/original task estimates may fail
 - continuous adaptation
 - ➔ launch a “good enough” product



Organizing ML projects

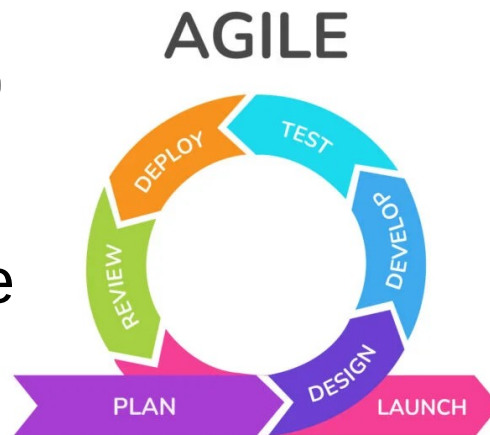
- **agile** workflow: (agile manifesto, description)

- ➔ pros:

- flexible, self-improving, communicative
- customer-centric

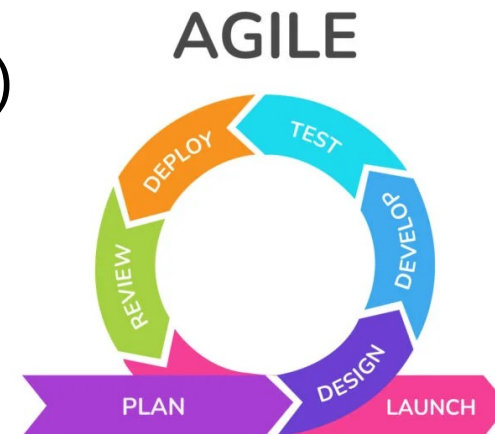
- ➔ cons:

- over-emphasized meetings (“agile theater”)
- over-controlled
- less predictable



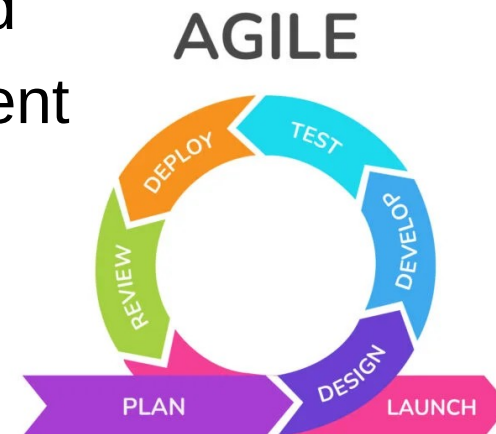
Organizing ML projects

- **agile** workflow: (**agile manifesto**, **description**)
 - ➔ loops, sprints
 - ➔ user interface/user experience (UI/UX)
 - ➔ CI/CD, CI/CD pipelines
continuous integration, continuous deployment
 - ➔ Kanban/Jira: project development visualization
(backlog, to do, in progress, testing, done)



Organizing ML projects

- typical small group agile project
 - define the goal (user stories), create a backlog (list of tasks)
 - make repo (github), branches
 - make MVP, backend API, frontend, testbed
 - get MVP working, deploy to test environment
 - collect feedback (team + users)
 - refine tasks, and directions to proceed



Structure of a professional ML organization

-  Business side
 - ➔ client/stakeholder: business requirements (e.g., “reduce equipment downtime,” “recommend better products”)
 - ➔ Business Analyst / Product Manager (translates business goals into ML problems, acts as the bridge between client and technical teams)
 - ➔ Project/ Product Manager (budgeting, timelines, and resource allocation, Coordinates with business analysts and technical leads to estimate cost)

Production level ML organization

-  Data/model experts
 - ➔ data engineer: build and maintain data pipelines → data collection, cleaning, validating, transformation (ETL/ELT), storing (database, data lake), monitoring
 - ➔ ML engineer/data scientist
 - feature generation
 - design, train, and fine-tune ML models
 - explores and experiments with data, build prototypes
 - performance evaluation

Production level ML organization



Software & Infrastructure

- ➔ software architect: designs overall system, service planning, integration with other services, APIs, databases
- ➔ software developer (coder): implements algorithms, frontend-backend
- ➔ DevOps/MLOps engineer:
 - automates deployment, monitoring, scaling of ML models
 - works with Docker, Kubernetes, CI/CD pipelines, cloud platforms.
 - ensures the ML system is reproducible, reliable, and easy to maintain.

Production level ML organization

-  Operations, Monitoring & Safety
 - ➔ QA / Test Engineer: tests software and ML pipelines (unit tests, integration tests, stress tests), ensures reliability before release
 - ➔ Monitoring & Reliability Engineer: monitors system health after deployment.; tracks data drift, performance degradation, unusual patterns
 - ➔ Security Engineer: ensures compliance with data privacy, protects against adversarial attacks, model leaks.

The Full Pipeline in a Story (user story)

- Client says: “I need early warnings for dangers in a chemical plant.”
- Business Analyst refines: “This means anomaly detection on sensor signals.”
- Data Engineer sets up pipelines to collect sensor data.
- Data Scientist explores, builds a prototype anomaly detector.
- ML Engineer optimizes the model for latency and accuracy.



The Full Pipeline in a Story (user story)

- Software Architect decides how it fits into the production system.
- Developers implement the service around the model.
- DevOps/MLOps deploys it to the cloud with monitoring.
- QA & Monitoring check performance and flag issues.
- Security Engineer ensures compliance and robustness.

END OF LECTURE 2